



UNIVERSIDAD DE  
COSTA RICA

SP-CIE

Carrera de  
Informática Empresarial  
Sede del Pacífico



Universidad de Costa Rica Sede  
Regional del Pacífico Bach.  
Informática Empresarial  
Desarrollo de Software III  
Fecha de entrega: **1 de junio del 2026**

## Proyecto Programado

**Valor:** 10%

**Modalidad:** Grupal (3 estudiantes)

**Duración:** 1 mes

## Nombre del proyecto

**Sistema de Software Aplicado a un Problema de Interés Social**

## Objetivo del proyecto

El objetivo de este proyecto es que los estudiantes diseñen, desarrollen y documenten una aplicación de software completa que resuelva un problema de interés social, aplicando:

- arquitectura **MVC**
- principios **SOLID**
- uso de **estructuras de datos**
- persistencia en **JSON**
- desarrollo de una **GUI modular**
- implementación de **login, procesos y reportes**
- uso de **control de versiones con Git y GitHub**



El proyecto debe reflejar tanto funcionalidad como una **estructura de software bien diseñada**, similar a entornos reales de desarrollo.

## Selección del problema

Cada grupo debe proponer un sistema basado en un **problema de interés social**, por ejemplo:

- gestión de donaciones
- sistema de voluntariado
- control de inventario en comedores sociales
- gestión de citas médicas comunitarias
- adopción de mascotas
- seguimiento de estudiantes en programas educativos

El tema debe ser aprobado por el docente.

## Requerimientos del sistema

El sistema debe incluir obligatoriamente:

### 1. Módulo de autenticación (Login)

El sistema debe permitir:

- registro de usuarios (opcional)
- inicio de sesión
- validación de credenciales



- acceso restringido a funcionalidades

Los datos pueden almacenarse en JSON.

## 2. Mínimo 3 módulos funcionales

El sistema debe incluir al menos tres módulos relacionados con el problema.

Cada módulo debe permitir:

- registro de datos
- consulta individual
- listado en tabla

Ejemplo:

- módulo de usuarios
- módulo de recursos
- módulo de procesos

## 3. Procesos del sistema

El sistema debe incluir lógica de negocio, por ejemplo:

- cálculos
- validaciones
- relaciones entre entidades
- generación de resultados

Esto debe implementarse en la **capa de servicios**.



## 4. Reportes

El sistema debe incluir al menos **dos reportes**, como:

- listado general
- consultas filtradas
- estadísticas o totales

Los reportes deben visualizarse en la GUI.

## 5. Persistencia

Debe implementarse:

- almacenamiento en archivos JSON
- carga automática de datos al iniciar
- archivos separados por entidad

## 6. Interfaz gráfica

Debe incluir:

- navegación entre módulos
- formularios
- tablas para consultas

La GUI debe estar **separada en módulos (no una sola clase)**.



# Arquitectura obligatoria

El sistema debe estructurarse en:

- **Model**
- **Repository**
- **Service**
- **Controller**
- **View (GUI modular)**

# Aplicación de SOLID

El sistema debe demostrar:

- separación de responsabilidades
- bajo acoplamiento
- código reutilizable

# Uso de estructuras de datos

Debe evidenciar uso de:

- listas
- diccionarios
- tuplas
- conjuntos



# Uso obligatorio de Git y GitHub

Cada grupo debe trabajar utilizando control de versiones.

## Requisitos obligatorios:

1. Crear un repositorio en GitHub
2. Subir el proyecto completo
3. Trabajar en equipo utilizando Git

## Condiciones mínimas

- El repositorio debe ser compartido entre los 3 integrantes
- Cada estudiante debe realizar commits
- Deben existir **mínimo 5 commits en total**

## Lo que se evaluará en GitHub

- historial de commits
- claridad de los mensajes
- evidencia de trabajo colaborativo
- evolución del proyecto



# Plan de trabajo detallado

## Semana 1 — Análisis y diseño

El grupo debe:

- definir el problema
- describir el sistema
- identificar módulos
- definir entidades principales
- diseñar el sistema (UML)
- crear el repositorio en GitHub

## Semana 2 — Base del sistema

El grupo debe:

- implementar modelo (clases)
- implementar repositorios
- implementar persistencia JSON
- realizar primeros commits

## Semana 3 — Lógica del sistema

El grupo debe:

- implementar servicios (lógica)
- implementar controlador
- desarrollar procesos del sistema
- continuar commits



## Semana 4 — Interfaz y cierre

El grupo debe:

- desarrollar GUI modular
- implementar reportes
- integrar login
- realizar pruebas
- documentar proyecto

## Entregable 1 — Adelanto

Debe incluir:

### Documento

- nombre del proyecto
- problema
- justificación
- módulos
- diagrama UML
- estructura del sistema



# Entregable final

Debe incluir:

- código completo
- GUI funcional
- persistencia funcionando
- repositorio en GitHub
- mínimo 5 commits
- README.md con:
  - descripción
  - arquitectura
  - SOLID
  - instrucciones de uso

# Restricciones

- no usar frameworks
- no hacer una sola clase gigante
- no mezclar lógica con GUI
- no duplicar código



## Evaluación (10%)

| Criterio                  | Valor |
|---------------------------|-------|
| Funcionalidad del sistema | 3%    |
| Arquitectura MVC          | 2%    |
| Principios SOLID          | 2%    |
| GUI modular               | 1.5%  |
| Persistencia JSON         | 1%    |
| GitHub y commits          | 0.5%  |

## Resultado esperado

El estudiante debe demostrar que puede:

- ✓ diseñar un sistema
- ✓ estructurar software correctamente
- ✓ trabajar en equipo con Git
- ✓ implementar lógica real
- ✓ construir una GUI funcional

“No se evalúa solo que funcione... se evalúa cómo está construido.”